

Introduction to Algorithms

Topic 3: Comparison-Based Sorting Algorithms

Xiang-Yang Li and Haisheng Tan¹

School of Computer Science and Technology
University of Science and Technology of China (USTC)

Fall Semester 2026

Outline

Basic Concepts

Simple Sorting Algorithms

Efficient Sorting Algorithms

Summary

Basic Concepts of Sorting Algorithm

Stability

A sorting algorithm is stable if records with equal keys retain their relative order after sorting; otherwise, it is unstable.

Example: $2, 2^*, 1 \rightarrow 1, 2^*, 2$ (unstable sorting)

Basic Concepts of Sorting Algorithm

Stability

A sorting algorithm is stable if records with equal keys retain their relative order after sorting; otherwise, it is unstable.

Example: $2, 2^*, 1 \rightarrow 1, 2^*, 2$ (unstable sorting)

Time Complexity

Usually measured by the number of data **comparisons** and the number of data **movements** in the algorithm execution.

Basic Concepts of Sorting Algorithm

Stability

A sorting algorithm is stable if records with equal keys retain their relative order after sorting; otherwise, it is unstable.

Example: $2, 2^*, 1 \rightarrow 1, 2^*, 2$ (unstable sorting)

Time Complexity

Usually measured by the number of data **comparisons** and the number of data **movements** in the algorithm execution.

In-place Sorting

An in-place sorting algorithm rearranges elements within the input array using $O(1)$ auxiliary array storage. Any recursion stack is reported separately.

Contents

Basic Concepts

Simple Sorting Algorithms

Insertion Sort

Selection Sort

Bubble Sort

Efficient Sorting Algorithms

Summary

Insertion Sort

General idea: Maintain an ordered sequence.

Insertion Sort

General idea: Maintain an ordered sequence.

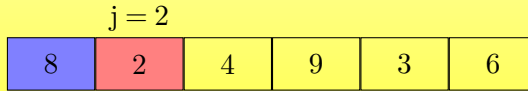
Insertion-Sort(A)

```
1: for  $j = 2$  to  $A.length$  do
2:    $key = A[j]$ 
3:   // Insert  $A[j]$  into the sorted sequence  $A[1..j - 1]$ .
4:    $i = j - 1$ 
5:   while  $i > 0$  and  $A[i] > key$  do
6:      $A[i + 1] = A[i]$ 
7:      $i = i - 1$ 
8:    $A[i + 1] = key$ 
```

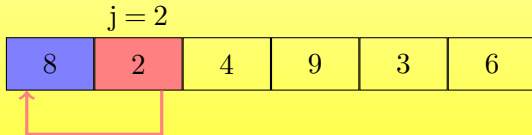
Example of Insertion Sort

8	2	4	9	3	6
---	---	---	---	---	---

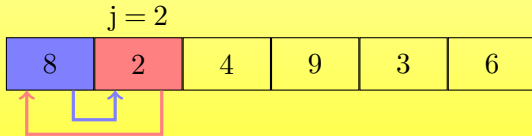
Example of Insertion Sort



Example of Insertion Sort



Example of Insertion Sort

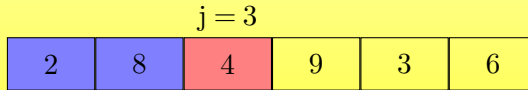


Example of Insertion Sort

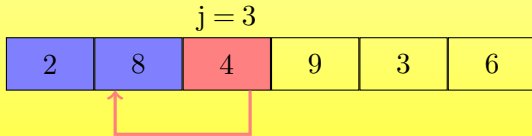
$j = 3$

2	8	4	9	3	6
---	---	---	---	---	---

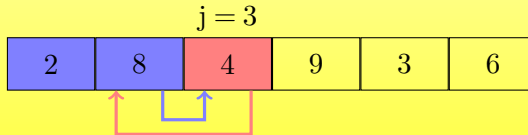
Example of Insertion Sort



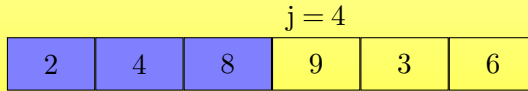
Example of Insertion Sort



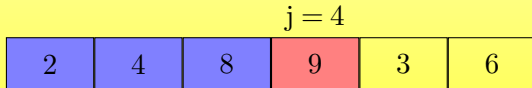
Example of Insertion Sort



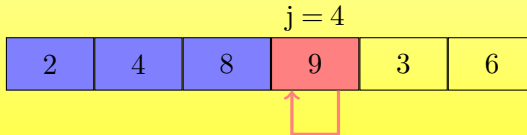
Example of Insertion Sort



Example of Insertion Sort



Example of Insertion Sort

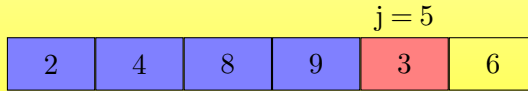


Example of Insertion Sort

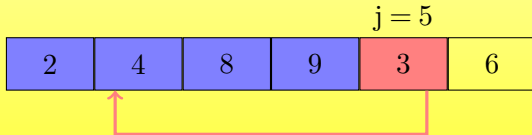
$j = 5$

2	4	8	9	3	6
---	---	---	---	---	---

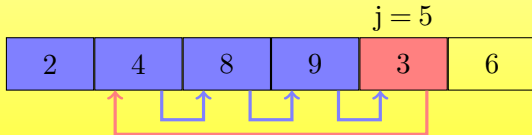
Example of Insertion Sort



Example of Insertion Sort



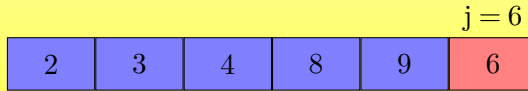
Example of Insertion Sort



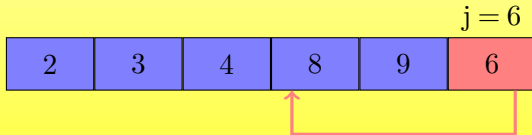
Example of Insertion Sort



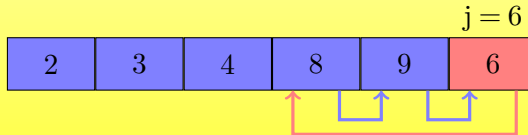
Example of Insertion Sort



Example of Insertion Sort



Example of Insertion Sort



Example of Insertion Sort

2	3	4	6	8	9
---	---	---	---	---	---

Insertion Sort

- ▶ Time Complexity
 - ▶ Best: $O(n)$
 - ▶ Average: $O(n^2)$
 - ▶ Worst: $O(n^2)$
- ▶ Auxiliary space: $O(1)$
- ▶ Stable: Yes

Insertion Sort

- ▶ Time Complexity
 - ▶ Best: $O(n)$
 - ▶ Average: $O(n^2)$
 - ▶ Worst: $O(n^2)$
- ▶ Auxiliary space:
 $O(1)$
- ▶ Stable: Yes

Insertion-Sort(A)

```
1: for j = 2 to A.length do
2:   key = A[j]
3:   // Insert A[j] into the sorted
   sequence A[1..j - 1].
4:   i = j - 1
5:   while i > 0 and A[i] > key do
6:     A[i + 1] = A[i]
7:     i = i - 1
8:   A[i + 1] = key
```

Contents

Basic Concepts

Simple Sorting Algorithms

Insertion Sort

Selection Sort

Bubble Sort

Efficient Sorting Algorithms

Summary

Selection Sort

General idea: Select the minimum element from the unsorted suffix and place it at the next position.

Selection Sort

General idea: Select the minimum element from the unsorted suffix and place it at the next position.

Selection-Sort(A)

- 1: for $i = 1$ to $A.length - 1$ do
- 2: $k = i$ ▷ k is the position of the **smallest** key.
- 3: for $j = i + 1$ to $A.length$ do
- 4: if $A[j] < A[k]$ then
- 5: $k = j$
- 6: if $k \neq i$ then
- 7: $A[i] \leftrightarrow A[k]$

Example of Selection Sort

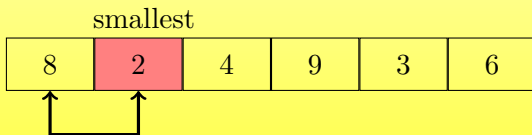
8	2	4	9	3	6
---	---	---	---	---	---

Example of Selection Sort

smallest

8	2	4	9	3	6
---	---	---	---	---	---

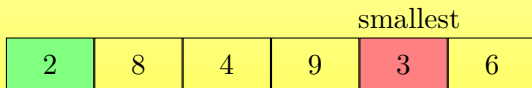
Example of Selection Sort



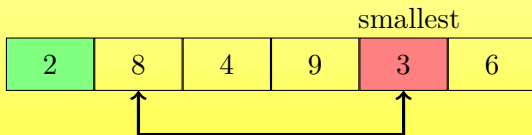
Example of Selection Sort

2	8	4	9	3	6
---	---	---	---	---	---

Example of Selection Sort



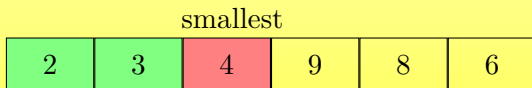
Example of Selection Sort



Example of Selection Sort

2	3	4	9	8	6
---	---	---	---	---	---

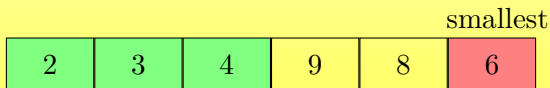
Example of Selection Sort



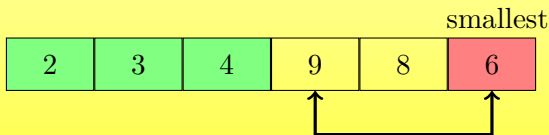
Example of Selection Sort

2	3	4	9	8	6
---	---	---	---	---	---

Example of Selection Sort



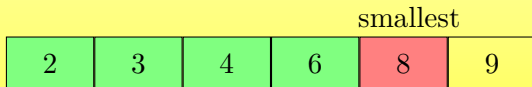
Example of Selection Sort



Example of Selection Sort

2	3	4	6	8	9
---	---	---	---	---	---

Example of Selection Sort



Example of Selection Sort

2	3	4	6	8	9
---	---	---	---	---	---

Selection Sort

- ▶ Time Complexity
 - ▶ Best: $O(n^2)$
 - ▶ Average: $O(n^2)$
 - ▶ Worst: $O(n^2)$
- ▶ Auxiliary space:
 $O(1)$
- ▶ Stable: No

Selection Sort

- ▶ Time Complexity
 - ▶ Best: $O(n^2)$
 - ▶ Average: $O(n^2)$
 - ▶ Worst: $O(n^2)$
- ▶ Auxiliary space:
 $O(1)$
- ▶ Stable: No

Selection-Sort(A)

```
1: for i = 1 to A.length - 1 do
2:     k = i
3:     for j = i + 1 to A.length do
4:         if A[j] < A[k] then
5:             k = j
6:     if k ≠ i then
7:         A[i] ↔ A[k]
```

Selection Sort

Stable sorting: How to revise the selection sorting to make it stable?

Contents

Basic Concepts

Simple Sorting Algorithms

Insertion Sort

Selection Sort

Bubble Sort

Efficient Sorting Algorithms

Summary

Bubble Sort

General idea: From the back to the front, if some elements are smaller than their predecessor, then swap them.

Bubble Sort

General idea: From the back to the front, if some elements are smaller than their predecessor, then swap them.

Bubble-Sort(A)

```
1: for  $i = 1$  to  $A.length - 1$  do
2:   noswap = TRUE
3:   for  $j = A.length - 1$  downto  $i$  do
4:     if  $A[j + 1] < A[j]$  then
5:        $A[j] \leftrightarrow A[j + 1]$ 
6:       noswap = FALSE
7:   if noswap then break
```

Example of Bubble Sort

8	2	4	9	3	6
---	---	---	---	---	---

Example of Bubble Sort

i						$j+1$
8	2	4	9	3	6	

Example of Bubble Sort

i		$j+1$			
8	2	4	9	3	6

Example of Bubble Sort

i		$j+1$			
8	2	4	3	9	6

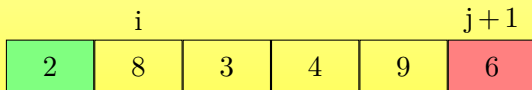
Example of Bubble Sort

i		$j + 1$			
8	2	3	4	9	6

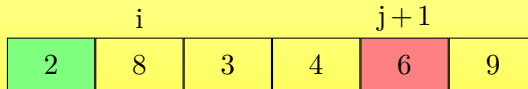
Example of Bubble Sort

i	j + 1				
8	2	3	4	9	6

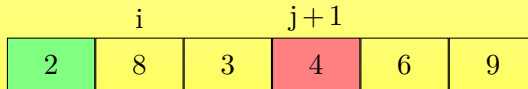
Example of Bubble Sort



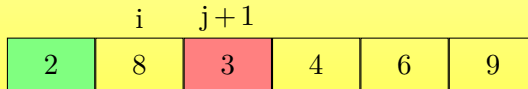
Example of Bubble Sort



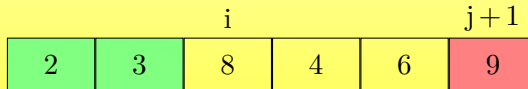
Example of Bubble Sort



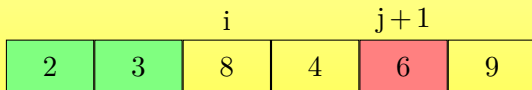
Example of Bubble Sort



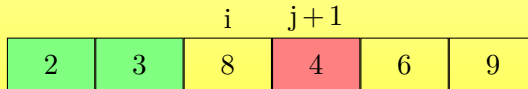
Example of Bubble Sort



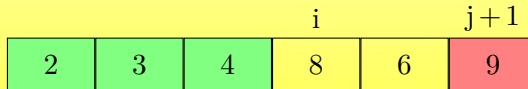
Example of Bubble Sort



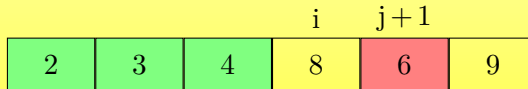
Example of Bubble Sort



Example of Bubble Sort



Example of Bubble Sort



Example of Bubble Sort

2	3	4	6	8	9
---	---	---	---	---	---

Bubble Sort

- ▶ Time Complexity
 - ▶ Best: $O(n)$
 - ▶ Average: $O(n^2)$
 - ▶ Worst: $O(n^2)$
- ▶ Auxiliary space:
 $O(1)$
- ▶ Stable: Yes

Bubble Sort

- ▶ Time Complexity
 - ▶ Best: $O(n)$
 - ▶ Average: $O(n^2)$
 - ▶ Worst: $O(n^2)$
- ▶ Auxiliary space:
 $O(1)$
- ▶ Stable: Yes

Bubble-Sort(A)

```
1: for  $i = 1$  to  $A.length - 1$  do
2:     noswap = TRUE
3:     for  $j = A.length - 1$  downto  $i$  do
4:         if  $A[j + 1] < A[j]$  then
5:              $A[j] \leftrightarrow A[j + 1]$ 
6:             noswap = FALSE
7:     if noswap then break
```

Contents

Basic Concepts

Simple Sorting Algorithms

Efficient Sorting Algorithms

Shellsort

Heapsort

Quicksort

Summary

Shellsort

General idea:

- ▶ Choose a descending gap sequence (e.g., $D = [5, 3, 2, 1]$).
- ▶ In each round, elements with the same gap d are in the same group.
- ▶ Apply Insertion-Sort for each group.
- ▶ Reduce the amount of data migration that caused by insertion sort.

Shellsort

Shell-Pass(A, d)

- 1: for $i = d + 1$ to n do
- 2: if $A[i] < A[i - d]$ then
- 3: $key = A[i]$ // $A[i]$ is to inserted in the correct position
- 4: $j = i - d$
- 5: while $j > 0$ and $key < A[j]$ do
- 6: $A[j + d] = A[j]$
- 7: $j = j - d$
- 8: $A[j + d] = key$

Shellsort(A, D)

- 1: for increment in D do
- 2: Shell-Pass($A, increment$)

Example of Shellsort

21 25 49 25 16 08 27 04 55 48

Example of Shellsort

21 25 49 25 16 08 27 04 55 48 $d = 3$

Example of Shellsort

21 25 49 25 16 08 27 04 55 48 $d = 3$

21 04 08 25 16 49 27 25 55 48

Example of Shellsort

21 25 49 25 16 08 27 04 55 48 $d = 3$

21 04 08 25 16 49 27 25 55 48 $d = 2$

Example of Shellsort

21 25 49 25 16 08 27 04 55 48 $d = 3$

21 04 08 25 16 49 27 25 55 48 $d = 2$

08 04 16 25 21 25 27 48 55 49

Example of Shellsort

21 25 49 25 16 08 27 04 55 48 $d = 3$

21 04 08 25 16 49 27 25 55 48 $d = 2$

08 04 16 25 21 25 27 48 55 49 $d = 1$

Example of Shellsort

21 25 49 25 16 08 27 04 55 48 $d = 3$

21 04 08 25 16 49 27 25 55 48 $d = 2$

08 04 16 25 21 25 27 48 55 49 $d = 1$

04 08 16 21 25 25 27 48 49 55

Shellsort

- ▶ Time Complexity
 - ▶ Best: depends on the gap sequence
 - ▶ Average: depends on the gap sequence
 - ▶ Worst: depends on the gap sequence, e.g., $O(n^{4/3})$, for $4^k + 3 \cdot 2^{k-1} + 1$ ($k \geq 1$), prefixed with 1.
- ▶ Auxiliary space: $O(1)$
- ▶ Stable: No

Shellsort

Shellsort

Why Shellsort typically performs faster?

- ▶ Insertion-Sorting small-sized array although costs $O(n^2)$ in the worst case, but it is similar to $O(n)$ in values.
- ▶ For large array, when we use a gap large enough (in the order of $O(n)$), each sub-array has a small size, thus efficient to sort.
- ▶ After enough iterations, when the gap is small, the majority part of the array is already sorted (thus the complexity is small again).

Shellsort

How to select the gap sequence?

- ▶ $\lceil \frac{n}{2^k} \rceil$: time complexity $\Theta(n^2)$
- ▶ $2 \lfloor \frac{n}{2^{k+1}} \rfloor + 1$: time complexity $\Theta(n^{\frac{3}{2}})$
- ▶ $2^k - 1$: time complexity $\Theta(n^{\frac{3}{2}})$
- ▶ $2^k + 1$ ($k \geq 1$), prefixed with 1: time complexity $\Theta(n^{\frac{3}{2}})$
- ▶ Successive numbers of the form $2^p 3^q$, where $p, q \in \mathbb{Z}_{\geq 0}$: time complexity $O(n \log^2 n)$.

Shellsort: the lowerbound on the time-complexity

The worst-case complexity of any version of Shellsort is of higher order: Plaxton, Poonen, and Suel showed that it grows at least as rapidly as $\Omega\left(n\left(\frac{\log n}{\log \log n}\right)^2\right)$.

Contents

Basic Concepts

Simple Sorting Algorithms

Efficient Sorting Algorithms

Shellsort

Heapsort

Quicksort

Summary

Basic Concepts of Heap

Heap

A data structure which is an array object that can be viewed as a **nearly complete binary tree**.

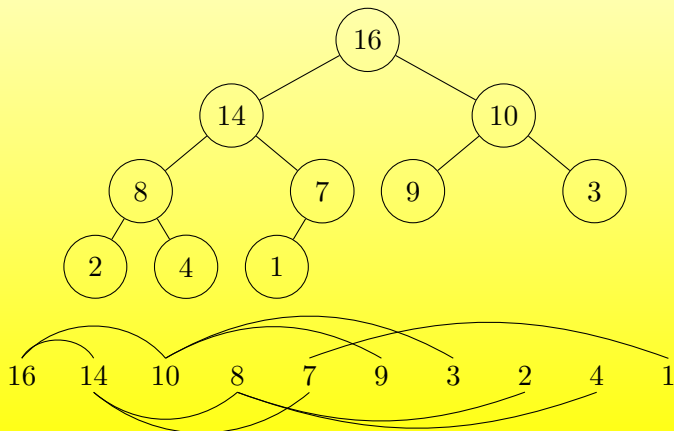
The tree is completely filled on all levels except possibly the lowest, which is filled from the left up to a point.

Given the index i of a node, the indices of its parent $\text{Parent}(i)$, left child $\text{Left}(i)$, and right child $\text{Right}(i)$ can be computed simply:

$\text{Parent}(i)$	return $\lfloor i/2 \rfloor$
$\text{Left}(i)$	return $2*i$
$\text{Right}(i)$	return $2*i + 1$

Example of Max-heap

max-heap: $A[\text{Parent}(i)] \geq A[i]$, for all i other than the root.



Maintaining the Heap Property

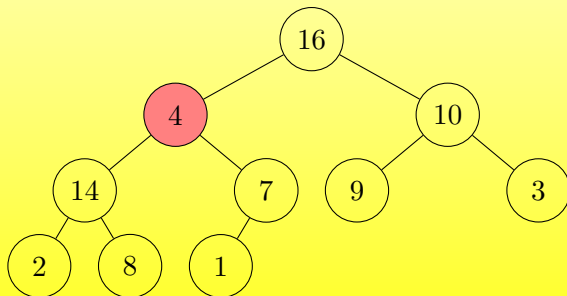
Assumption: sub-trees rooted at $\text{Left}(i)$ & $\text{Right}(i)$ are max-heaps.

Max-Heapify(A, i) // Input an an array and an index i

- 1: $l = \text{Left}(i)$;
- 2: $r = \text{Right}(i)$
- 3: if $l \leq A.\text{heap-size}$ and $A[l] > A[i]$ then
- 4: $\text{largest} = l$
- 5: else $\text{largest} = i$
- 6: if $r \leq A.\text{heap-size}$ and $A[r] > A[\text{largest}]$ then
- 7: $\text{largest} = r$
- 8: if $\text{largest} \neq i$ then
- 9: $A[i] \leftrightarrow A[\text{largest}]$
- 10: Max-Heapify($A, \text{largest}$)

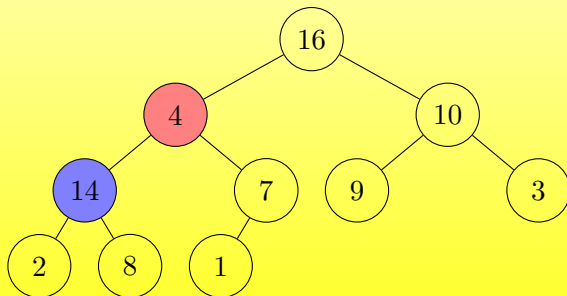
Maintaining the Heap Property

Example: MAX-HEAPIFY(A, 2)



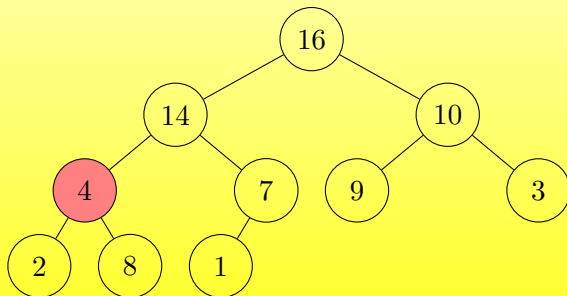
Maintaining the Heap Property

Example: MAX-HEAPIFY(A, 2)



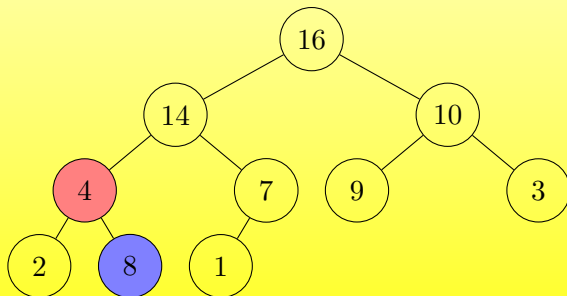
Maintaining the Heap Property

Example: MAX-HEAPIFY(A, 2)



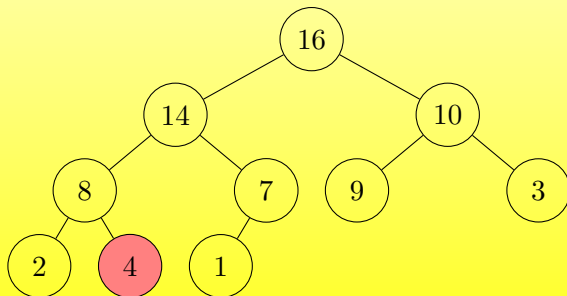
Maintaining the Heap Property

Example: MAX-HEAPIFY(A, 2)



Maintaining the Heap Property

Example: MAX-HEAPIFY(A, 2)



Maintaining the Heap Property

Assumption: sub-trees rooted at $\text{Left}(i)$ & $\text{Right}(i)$ are max-heaps.

Max-Heapify(A, i) // Input an an array and an index i

- 1: $l = \text{Left}(i)$;
- 2: $r = \text{Right}(i)$
- 3: if $l \leq A.\text{heap-size}$ and $A[l] > A[i]$ then
- 4: $\text{largest} = l$
- 5: else $\text{largest} = i$
- 6: if $r \leq A.\text{heap-size}$ and $A[r] > A[\text{largest}]$ then
- 7: $\text{largest} = r$
- 8: if $\text{largest} \neq i$ then
- 9: $A[i] \leftrightarrow A[\text{largest}]$
- 10: Max-Heapify($A, \text{largest}$)

Building a Heap

Fact: with the array representation of an n -element heap, the leaves are the nodes indexed from $\lfloor A.length/2 \rfloor + 1$ to n , and each leaf is a 1-element max-heap to begin with.

Building a Heap

Fact: with the array representation of an n -element heap, the leaves are the nodes indexed from $\lfloor A.length/2 \rfloor + 1$ to n , and each leaf is a 1-element max-heap to begin with.

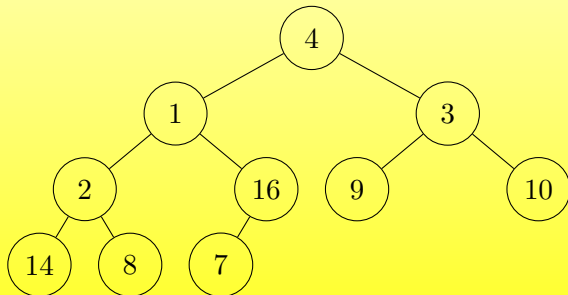
Build-Max-Heap(A)

- 1: $A.heap-size = A.length$
- 2: for $i = \lfloor A.length/2 \rfloor$ downto 1 do
- 3: Max-Heapify(A, i)

Building a Heap

A.length = 10

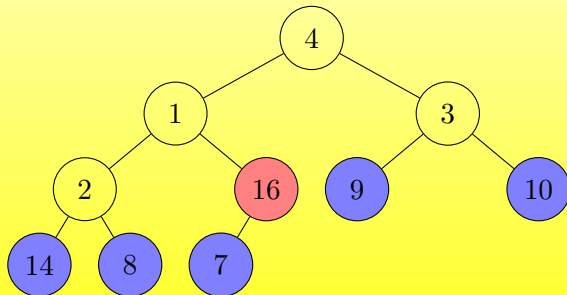
4 1 3 2 16 9 10 14 8 7



Building a Heap

A.length = 10

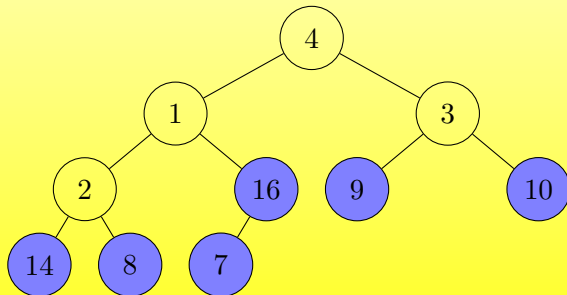
4 1 3 2 16 9 10 14 8 7



Building a Heap

A.length = 10

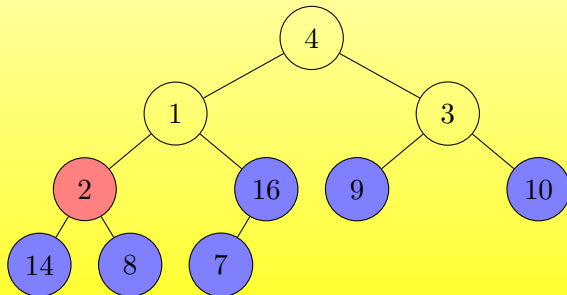
4 1 3 2 16 9 10 14 8 7



Building a Heap

A.length = 10

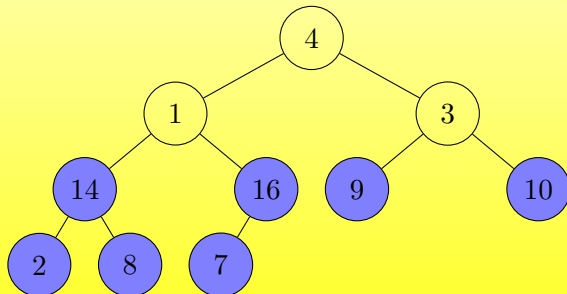
4 1 3 2 16 9 10 14 8 7



Building a Heap

A.length = 10

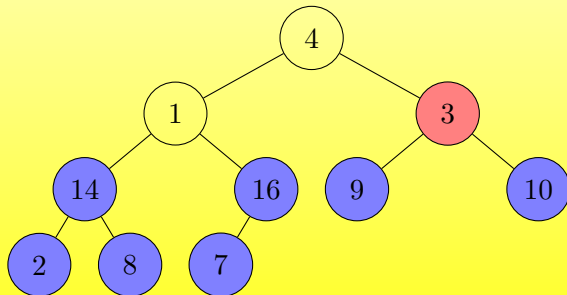
4 1 3 14 16 9 10 2 8 7



Building a Heap

A.length = 10

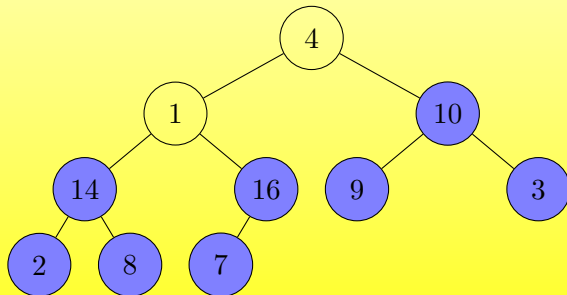
4 1 3 14 16 9 10 2 8 7



Building a Heap

A.length = 10

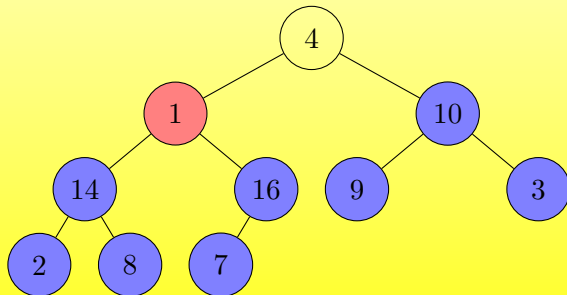
4 1 10 14 16 9 3 2 8 7



Building a Heap

A.length = 10

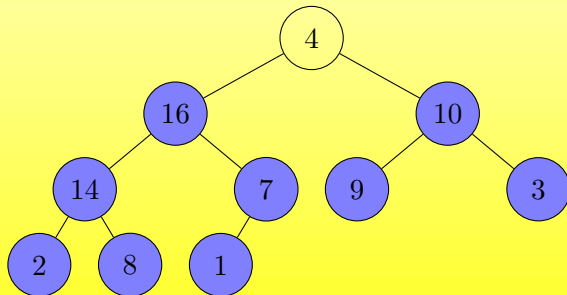
4 1 10 14 16 9 3 2 8 7



Building a Heap

A.length = 10

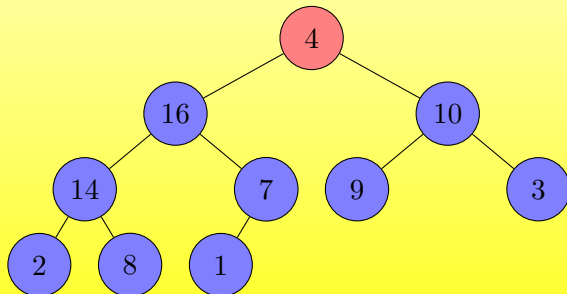
4 16 10 14 7 9 3 2 8 1



Building a Heap

A.length = 10

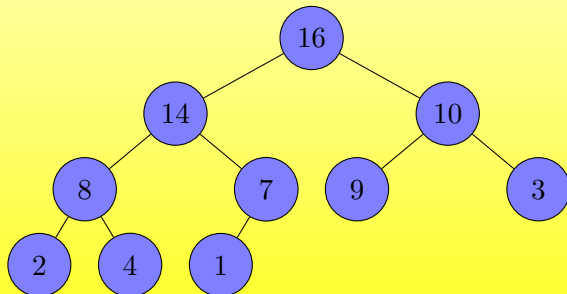
4 16 10 14 7 9 3 2 8 1



Building a Heap

A.length = 10

16 14 10 8 7 9 3 2 4 1



Building a Heap

Fact: with the array representation of an n -element heap, the leaves are the nodes indexed from $\lfloor A.length/2 \rfloor + 1$ to n , and each leaf is a 1-element max-heap to begin with.

Building a Heap

Fact: with the array representation of an n -element heap, the leaves are the nodes indexed from $\lfloor A.length/2 \rfloor + 1$ to n , and each leaf is a 1-element max-heap to begin with.

Build-Max-Heap(A)

- 1: $A.heap\text{-}size = A.length$
- 2: for $i = \lfloor A.length/2 \rfloor$ downto 1 do
- 3: Max-Heapify(A, i)

The Heapsort Algorithm

General idea: Same as selection sort, maintain the minimum (maximum) element by using heap.

MAX-HEAP: $A[1]$ always stores the largest number.

The Heapsort Algorithm

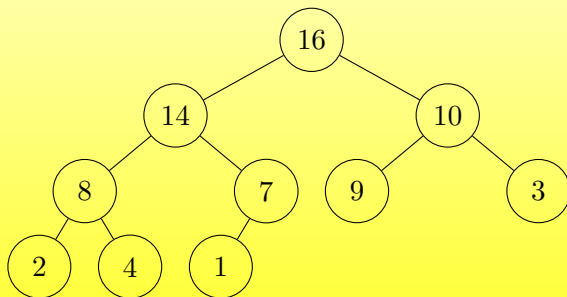
General idea: Same as selection sort, maintain the minimum (maximum) element by using heap.

MAX-HEAP: $A[1]$ always stores the largest number.

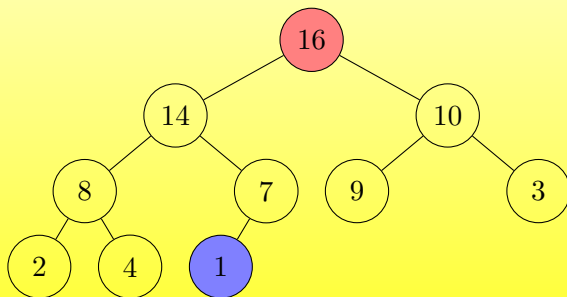
Heapsort(A)

- 1: Build-Max-Heap(A)
- 2: for $i = A.length$ downto 2 do
- 3: $A[1] \leftrightarrow A[i]$
- 4: $A.heap\text{-}size = A.heap\text{-}size - 1$
- 5: Max-Heapify($A, 1$)

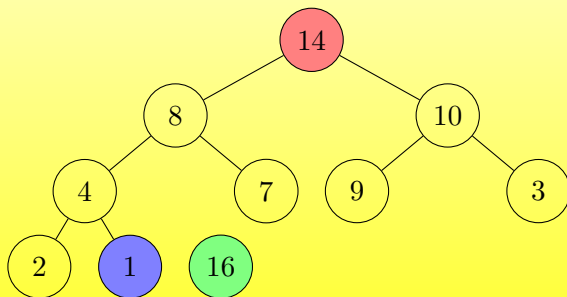
Example of Heapsort



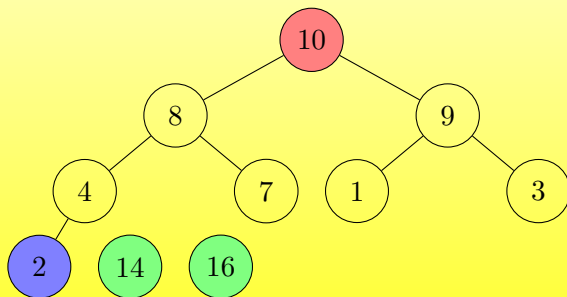
Example of Heapsort



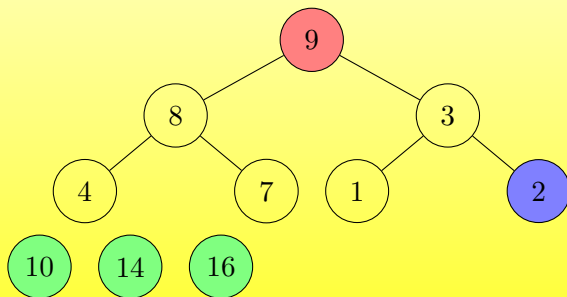
Example of Heapsort



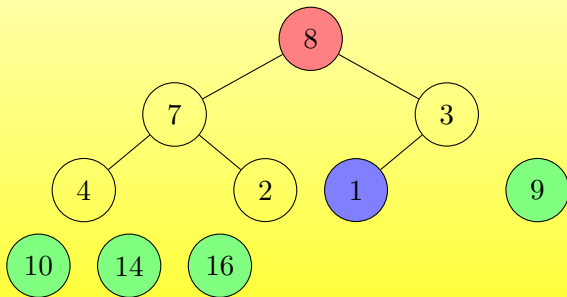
Example of Heapsort



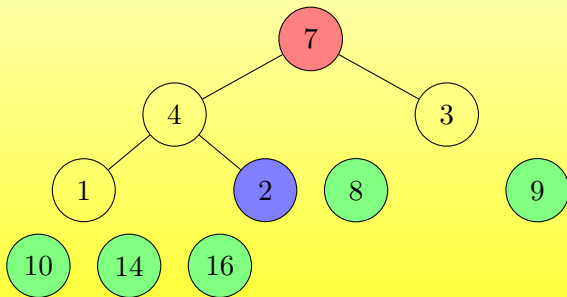
Example of Heapsort



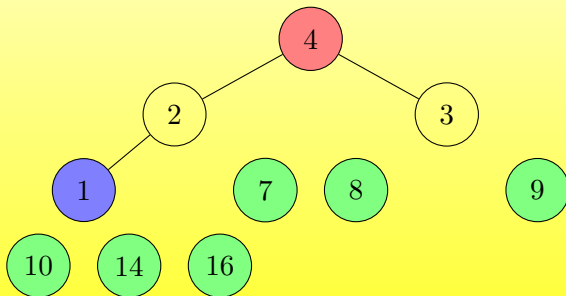
Example of Heapsort



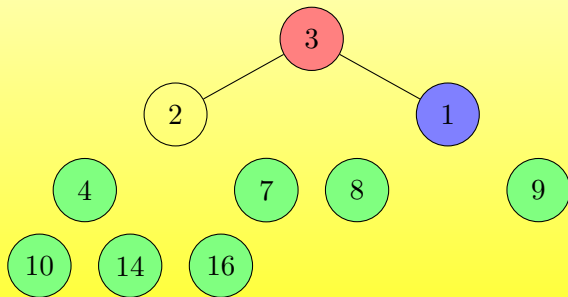
Example of Heapsort



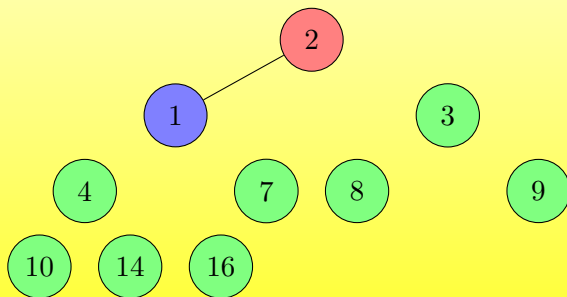
Example of Heapsort



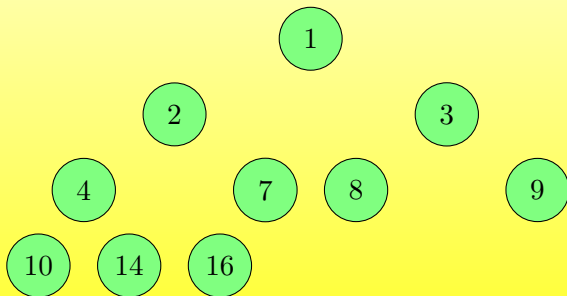
Example of Heapsort



Example of Heapsort



Example of Heapsort



Heapsort

- ▶ Time Complexity
 - ▶ Max-Heapify: $O(\log n)$ – Why?
 - ▶ Build-Max-Heap: $O(n)$ – Why?
 - ▶ Best: $O(n \log n)$
 - ▶ Average: $O(n \log n)$
 - ▶ Worst: $O(n \log n)$
- ▶ Auxiliary space: $O(\log n)$ for the recursive Max-Heapify shown; $O(1)$ with iterative heapify
- ▶ Stable: No

Priority Queues

A **priority queue** is a data structure for maintaining a set S of elements, each with an associated value called a **key**. A **max-priority queue** supports the following operations:

- ▶ $\text{Insert}(S, x)$ inserts the element x into the set S , which is equivalent to the operation $S = S \cup \{x\}$.
- ▶ $\text{Maximum}(S)$ returns the element of S with the largest key.
- ▶ $\text{Extract-Max}(S)$ removes and returns the element of S with the largest key.
- ▶ $\text{Increase-Key}(S, x, k)$ increases the value of element x 's key to the new value k , which is assumed to be at least as large as x 's current key value.

Priority Queues

Heap-Maximum(A)

1: return A[1]

Heap-Extract-Max(A)

- 1: if A.heap-size < 1 then
- 2: error “heap underflow”
- 3: max = A[1]
- 4: A[1] = A[A.heap-size]
- 5: A.heap-size =
 A.heap-size - 1
- 6: Max-Heapify(A, 1)
- 7: return max

Priority Queues

Heap-Increase-Key(A, i, key)

- 1: if $\text{key} < A[i]$ then
- 2: error “new key is smaller than current key”
- 3: $A[i] = \text{key}$
- 4: while $i > 1$ and $A[\text{Parent}(i)] < A[i]$ do
- 5: $A[i] \leftrightarrow A[\text{Parent}(i)]$
- 6: $i = \text{Parent}(i)$

Priority Queues

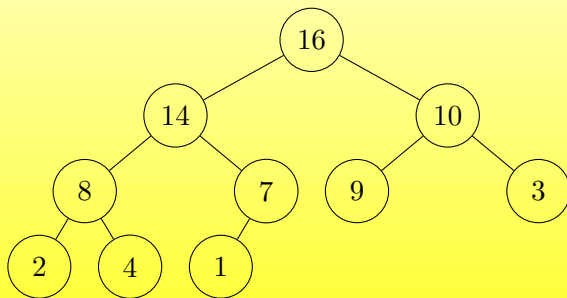
Max-Heap-Insert(A, key)

1: $A.\text{heap-size} = A.\text{heap-size} + 1$

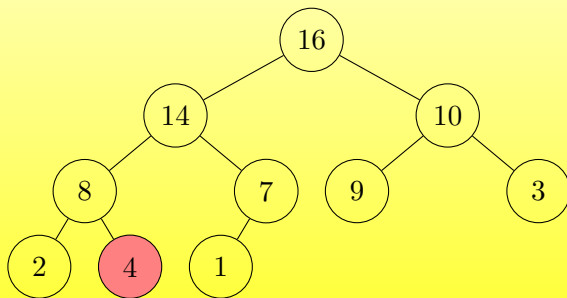
2: $A[A.\text{heap-size}] = -\infty$

3: Heap-Increase-Key($A, A.\text{heap-size}, \text{key}$)

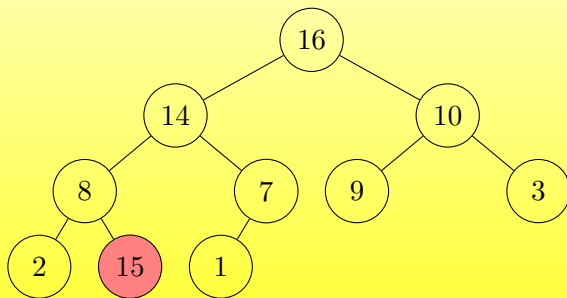
Example of Heap-Increase-Key



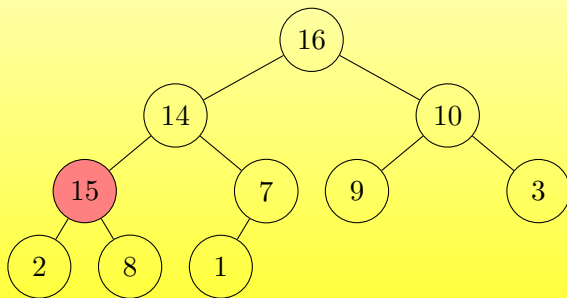
Example of Heap-Increase-Key



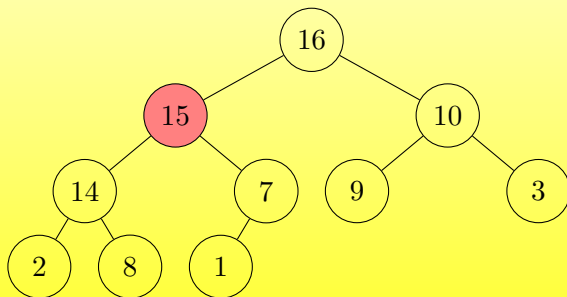
Example of Heap-Increase-Key



Example of Heap-Increase-Key



Example of Heap-Increase-Key



Priority Queues

Heap-Increase-Key(A, i, key)

- 1: if $\text{key} < A[i]$ then
- 2: error “new key is smaller than current key”
- 3: $A[i] = \text{key}$
- 4: while $i > 1$ and $A[\text{Parent}(i)] < A[i]$ do
- 5: $A[i] \leftrightarrow A[\text{Parent}(i)]$
- 6: $i = \text{Parent}(i)$

Contents

Basic Concepts

Simple Sorting Algorithms

Efficient Sorting Algorithms

Shellsort

Heapsort

Quicksort

Summary

Quicksort

General idea:

- ▶ **Arbitrarily choose** an element x in the unsorted set for comparison.
- ▶ **Divide** the unsorted elements into two parts: $\leq x$ and $> x$.
- ▶ **Recursively** use Quicksort for the above two parts.

Quicksort

General idea:

- ▶ **Arbitrarily choose** an element x in the unsorted set for comparison.
- ▶ **Divide** the unsorted elements into two parts: $\leq x$ and $> x$.
- ▶ **Recursively** use Quicksort for the above two parts.

Quicksort(A, p, r)

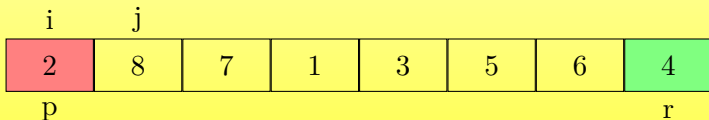
- 1: if $p < r$ then
- 2: $q = \text{Partition}(A, p, r)$
- 3: Quicksort($A, p, q - 1$)
- 4: Quicksort($A, q + 1, r$)

Partition

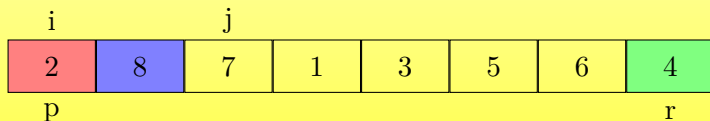
Partition(A, p, r)

- 1: $x = A[r]$ // pivot element
- 2: $i = p - 1$
- 3: for $j = p$ to $r - 1$ do
- 4: if $A[j] \leq x$ then
- 5: $i = i + 1$
- 6: $A[i] \leftrightarrow A[j]$
- 7: $A[i + 1] \leftrightarrow A[r]$
- 8: return $i + 1$

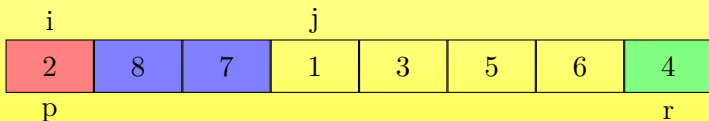
Example of Partition



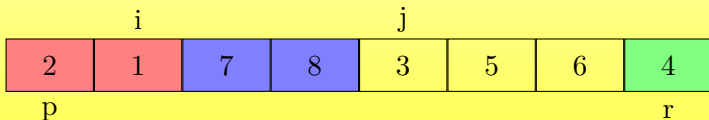
Example of Partition



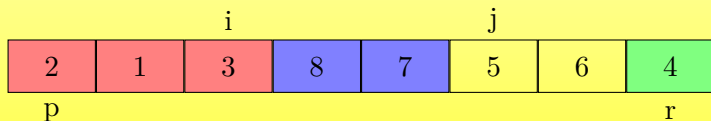
Example of Partition



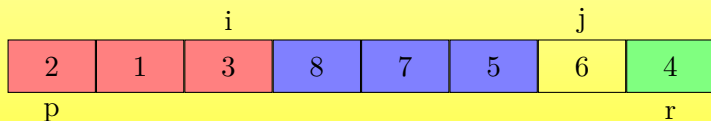
Example of Partition



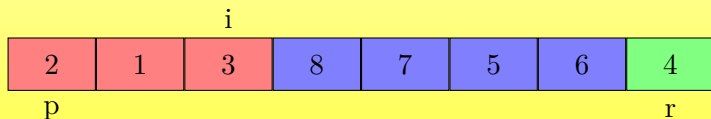
Example of Partition



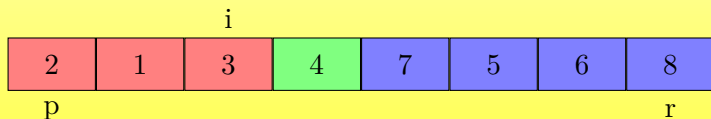
Example of Partition



Example of Partition



Example of Partition



Performance of Quicksort

Worst-case partitioning

The worst-case behavior for quicksort occurs when the partitioning routine produces one subproblem with $n - 1$ elements and one with 0 elements. The partitioning costs $\Theta(n)$ time. the recurrence for the running time is

$$\begin{aligned} T(n) &= T(n-1) + T(0) + \Theta(n) \\ &= T(n-1) + \Theta(n) \\ &= \Theta(n^2). \end{aligned}$$

Performance of Quicksort

Best-case partitioning

In the most even possible split, Partition produces two subproblems, each of size no more than $n/2$, since one is of size $\lfloor n/2 \rfloor$ and one of size $\lceil n/2 \rceil - 1$. In this case, quicksort runs much faster. The recurrence for the running time is then

$$\begin{aligned} T(n) &= 2T(n/2) + \Theta(n) \\ &= \Theta(n \lg n). \end{aligned}$$

Performance of Quicksort

Balanced partitioning

What if the split is always $\frac{1}{10} : \frac{9}{10}$? The recurrence for the running time is

$$\begin{aligned}T(n) &= T\left(\frac{1}{10}n\right) + T\left(\frac{9}{10}n\right) + \Theta(n) \\&= \Theta(n \lg n).\end{aligned}$$

A Randomized Version of Quicksort

Randomized-Partition(A, p, r)

- 1: $i = \text{Random}(p, r)$
- 2: $A[r] \leftrightarrow A[i]$
- 3: return Partition(A, p, r)

Randomized-Quicksort(A, p, r)

- 1: if $p < r$ then
- 2: $q = \text{Randomized-Partition}(A, p, r)$
- 3: Randomized-Quicksort($A, p, q - 1$)
- 4: Randomized-Quicksort($A, q + 1, r$)

Analysis of Quicksort

Worst-case analysis

A maximally unbalanced split at every level gives

$T(n) = T(n-1) + \Theta(n)$. Therefore, $T(n) = \Theta(\sum_{i=1}^n i) = \Theta(n^2)$.

The upper bound can also be verified by the [substitution method](#).

Analysis of Quicksort

Let $T(n)$ count partitioning work, with $T(0) = T(1) = 0$.
 For $n \geq 2$, suppose partitioning costs at most an for a constant
 $a > 0$. Assuming $T(k) \leq ck^2$ for $k < n$, we have

$$\begin{aligned} T(n) &\leq \max_{0 \leq q \leq n-1} \{T(q) + T(n-q-1)\} + an \\ &\leq \max_{0 \leq q \leq n-1} \{cq^2 + c(n-q-1)^2\} + an \\ &= c(n-1)^2 + an \\ &= cn^2 - (2c-a)n + c \\ &\leq cn^2 \quad (c \geq a, n \geq 1). \end{aligned}$$

The maximum is attained at $q = 0$ or $q = n-1$. Choose $c \geq a$
 large enough to cover the base cases.

Analysis of Quicksort

Running time and comparisons

Assume distinct keys and uniformly random pivots, and relabel the keys so that $z_1 < z_2 < \dots < z_n$. Let $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$. Define

$$X_{ij} = I\{z_i \text{ is compared to } z_j\}.$$

Since each pair is compared at most once, the **total number of comparisons** is

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}.$$

Analysis of Quicksort

$$\begin{aligned} E[X] &= E \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{z_i \text{ is compared to } z_j\} \end{aligned}$$

Analysis of Quicksort

$$\Pr\{X_{ij} = 1\} = \Pr\{z_i \text{ or } z_j \text{ is the first pivot in } Z_{ij}\} = \frac{2}{j-i+1},$$

$$\begin{aligned} E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{X_{ij} = 1\} \\ &= 2 \sum_{k=2}^n \frac{n+1-k}{k} = 2(n+1)H_n - 4n \\ &= \Theta(n \lg n), \end{aligned}$$

where $H_n = \sum_{k=1}^n 1/k$ is the n th harmonic number.

Quicksort

- ▶ Time Complexity
 - ▶ Best: $O(n \log n)$
 - ▶ Average: $O(n \log n)$
 - ▶ Worst: $O(n^2)$
- ▶ Auxiliary space: $O(\log n)$ expected; $O(n)$ worst case
- ▶ Stable: No

Summary

Name	Average	Worst	Stable	Method
Insertion Sort	$O(n^2)$	$O(n^2)$	Yes	Insertion
Selection Sort	$O(n^2)$	$O(n^2)$	No	Selection
Bubble Sort	$O(n^2)$	$O(n^2)$	Yes	Exchanging
Merge sort	$O(n \log n)$	$O(n \log n)$	Yes	Merging
Shellsort	(*)	$O(n^{4/3})$ (*)	No	Insertion
Heapsort	$O(n \log n)$	$O(n \log n)$	No	Selection
Quicksort	$O(n \log n)$	$O(n^2)$	No	Partitioning

*The time complexity of shellsort depends on the selected gap sequence.

A sorting algorithm animation website:

<https://www.toptal.com/developers/sorting-algorithms>